Linux网络--IO多路转接层04 LT与ET及epoll服务器实现

> ⓘ 系列导航
>
> 上一篇：Linux网络--IO多路转接层03 epoll事件模型与内核原理
> 系列索引：Linux网络--IO多路转接与Reactor系列总览
> 下一篇：Linux网络--Reactor模式层05 Connection回调与事件分发

- LT、ET 与 epoll 服务器实现
  - 1. 触发模式回答的是“通知几次”
  - 2. 用接收 2 KB 数据理解 LT
  - 3. 同一个例子在 ET 下怎样变化
  - 4. LT 与 ET
  - 5. ET 为什么必须非阻塞
  - 6. recv 返回值的完整解释
  - 7. ET 下 accept 也要排空
  - 8. 可写事件与部分发送
    - 8.1 EPOLLOUT 通常默认就绪
    - 8.2 send 可能只写一部分
    - 8.3 推荐状态机
  - 9. LT 版 epoll 服务器
    - 9.1 封装 Epoll
    - 9.2 服务器启动
    - 9.3 事件循环
  - 10. ET 版 epoll 服务器的修改点
    - 10.1 注册 ET
    - 10.2 非阻塞读接口
    - 10.3 非阻塞写接口
    - 10.4 ET 事件分支
  - 11. 为什么 ET 不一定“性能更高”
  - 12. 连接关闭的推荐顺序
  - 13. 本篇检查清单
  - 14. 从服务器循环到 Reactor

LT、ET 与 epoll 服务器实现

1. 触发模式回答的是“通知几次”

epoll 已经能把就绪事件放入队列，但还需要决定：

> 同一个 fd 的条件一直成立时，要不要反复报告？

LT 和 ET 就是两种答案：

- **LT，Level Triggered，水平触发**：条件持续成立，就持续通知；
- **ET，Edge Triggered，边缘触发**：状态出现新的边沿时通知。

epoll 默认使用 LT。给注册事件加上 `EPOLLET` 后，该 fd 使用 ET：

```
ev.events = EPOLLIN | EPOLLET;
```

2. 用接收 2 KB 数据理解 LT

假设接收缓冲区中有 2 KB 数据，应用收到 `EPOLLIN` 后只读取了 1 KB。

LT 下的下一次 `epoll_wait`：

```
接收缓冲区仍有 1 KB
        |
        v
“可读”条件仍成立
        |
        v
epoll_wait 再次返回 EPOLLIN
```

LT 允许应用暂时少读。应用处理慢一点不会立即失去后续机会，因为只要缓冲区里仍有数据，事件循环就能再次看到可读状态。

对应到服务器代码：

- 每次 `EPOLLIN` 可以读取一部分；
- 剩余数据由下一轮继续处理；
- 编程容错性较强；
- 如果应用长期不读，会增加重复唤醒。

3. 同一个例子在 ET 下怎样变化

第一次数据从“缓冲区为空”变为“有 2 KB 数据”，这是一个边沿，内核通知一次。

应用只读 1 KB后：

```
缓冲区仍有 1 KB
但状态没有产生新的边沿
        |
        v
epoll_wait 不再重复返回该连接
```

这里必须区分两件事：

- 数据仍在接收缓冲区，并没有被内核丢弃；
- 应用暂时失去了“继续处理旧数据”的通知。

如果此后客户端又写入新数据，接收状态发生变化，可能出现新的边沿，应用再次获得通知。但如果双方都在等待对方：

```
服务端：等客户端再发数据，触发下一次 EPOLLIN
客户端：等服务端处理完当前请求并返回响应
```

连接就会长期停住。ET 的主要风险不是数据丢失，而是应用没有在一次通知中完成本轮应做的排空工作。

4. LT 与 ET

注意三条关系：

1. **通知线与处理线不同**：内核通知“可读”和应用真正 `recv` 是两个动作。
2. **LT 看当前条件**：只要接收缓冲区仍有数据，条件就持续成立。
3. **ET 看状态变化**：只有新数据形成新边沿时才重新通知。

其中“只读 1KB、剩 1KB”的场景，不是让你背一个结论，而是用来发现 ET 的约束：应用不能把“这次只处理一部分”当成安全默认值。

5. ET 为什么必须非阻塞

ET 要求把本轮数据尽量读完。最简单的方式是循环读取：

```
while (true) {
    char buffer[4096];
    ssize_t n = recv(fd, buffer, sizeof(buffer), 0);

    if (n > 0) {
        input.append(buffer, n);
        continue;
    }

    if (n == 0) {
        close_connection(fd);
        break;
    }

    if (errno == EINTR) {
        continue;
    }

    if (errno == EAGAIN || errno == EWOULDBLOCK) {
        // 当前没有更多数据，本轮排空完成
        break;
    }

    // 真正的错误
    close_connection(fd);
    break;
}
```

最后一次读取时，应用通常不知道缓冲区还剩多少数据。数据读空后再调用 `recv`，如果 fd 是阻塞模式，调用会阻塞，整个事件循环随之停住。

所以工程上要使用非阻塞 fd：

```
bool SetNonBlocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    if (flags == -1) return false;

    return fcntl(fd, F_SETFL, flags | O_NONBLOCK) != -1;
}
```

非阻塞后，没有更多数据不会睡眠，而会返回 `EAGAIN` 或 `EWOULDBLOCK`。这两个错误在 ET 循环里表示正常结束。

6. recv 返回值的完整解释

返回值	含义	处理
`n > 0`	读到 `n` 字节	追加到输入缓冲区，继续按协议解析
`n == 0`	对端正常关闭读取方向	当前连接不再有更多请求，执行关闭流程
`n < 0, EINTR`	被信号打断	重试
`n < 0, EAGAIN/EWOULDBLOCK`	当前没有更多数据	ET 排空完成，结束循环
其他 `n < 0`	真实 I/O 错误	异常处理并关闭

这里的 `recv == 0` 不是“读到空字符串”，而是 TCP 对端发送了 FIN。

7. ET 下 accept 也要排空

监听 socket 使用 ET 时，一次 `EPOLLIN` 可能对应多个已经排队的连接。只调用一次 `accept`，就可能让 backlog 继续保留连接，却没有新的边沿提醒应用。

```
void AcceptLoop(int listen_fd) {
    while (true) {
        sockaddr_in peer;
        socklen_t len = sizeof(peer);

        int client_fd = accept(
            listen_fd,
            reinterpret_cast<sockaddr*>(&peer),
            &len
        );

        if (client_fd >= 0) {
            SetNonBlocking(client_fd);
            AddToEpoll(client_fd, EPOLLIN | EPOLLET);
            continue;
        }

        if (errno == EINTR) continue;
        if (errno == EAGAIN || errno == EWOULDBLOCK) break;
```

```
            perror("accept");
            break;
        }
    }
```

监听 fd 本身也要设置为非阻塞，否则 `accept` 在队列取空后可能阻塞。

8. 可写事件与部分发送

8.1 EPOLLOUT 通常默认就绪

刚建立连接时，发送缓冲区通常是空的，所以 `EPOLLOUT` 往往一直成立。如果永久注册写事件，`epoll_wait` 会持续返回，服务器不停执行没有业务意义的发送路径，形成忙循环。

写事件应只在有数据要发送且暂时发不出去时打开。

8.2 send 可能只写一部分

```
ssize_t n = send(fd, data, length, MSG_NOSIGNAL);
```

即使返回正数，也可能小于 `length`。应用必须记录已经发送的字节，并把剩余部分留在输出缓冲区。

8.3 推荐状态机

```
没有待发送数据
    -> 只关心 EPOLLIN

业务产生响应
    -> 追加到输出缓冲区
    -> 立即尝试 send

send 遇到 EAGAIN
    -> 保留剩余数据
    -> 改为关心 EPOLLIN | EPOLLOUT

下一次 EPOLLOUT
    -> 继续 send
```

```
输出缓冲区清空
    -> 删除输出缓冲区内容
    -> 恢复为只关心 EPOLLIN
```

使用 `MSG_NOSIGNAL`，或忽略 `SIGPIPE`，避免向已关闭连接发送数据时直接终止整个进程。

9. LT 版 epoll 服务器

9.1 封装 Epoll

```cpp
class Epoll {
public:
    Epoll() {
        epoll_fd_ = epoll_create1(EPOLL_CLOEXEC);
    }

    ~Epoll() {
        if (epoll_fd_ >= 0) {
            close(epoll_fd_);
        }
    }

    bool Add(int fd, uint32_t events) const {
        epoll_event ev = {0};
        ev.events = events;
        ev.data.fd = fd;
        return epoll_ctl(epoll_fd_, EPOLL_CTL_ADD, fd, &ev) == 0;
    }

    bool Del(int fd) const {
        return epoll_ctl(epoll_fd_, EPOLL_CTL_DEL, fd, nullptr) == 0;
    }

    int Wait(epoll_event* events, int maxevents, int timeout) const {
        return epoll_wait(epoll_fd_, events, maxevents, timeout);
    }

private:
    int epoll_fd_;
};
```

9.2 服务器启动

```
TcpSocket listen_sock;
listen_sock.Socket();
listen_sock.Bind(ip, port);
listen_sock.Listen(backlog);

Epoll epoll;
epoll.Add(listen_sock.GetFd(), EPOLLIN);
```

此时只有监听 fd 在关注集合中。服务器启动后并不直接读取客户端数据，因为还没有客户端连接。

9.3 事件循环

```
epoll_event events[1024];

for (;;) {
    int n = epoll.Wait(events, 1024, -1);
    if (n < 0) {
        if (errno == EINTR) continue;
        break;
    }

    for (int i = 0; i < n; ++i) {
        int fd = events[i].data.fd;

        if (fd == listen_sock.GetFd()) {
            TcpSocket client;
            if (listen_sock.Accept(&client, nullptr, nullptr)) {
                epoll.Add(client.GetFd(), EPOLLIN);
            }
            continue;
        }

        if (events[i].events & (EPOLLERR | EPOLLHUP)) {
            epoll.Del(fd);
            close(fd);
            continue;
        }

        if (events[i].events & EPOLLIN) {
            std::string request;
            if (!RecvAllAvailable(fd, &request)) {
                epoll.Del(fd);
```

```
                close(fd);
                continue;
            }

            std::string response;
            handler(request, &response);
            SendAllOrClose(fd, response);
        }
    }
}
```

LT 版可以每次先读取一批。由于可读条件仍成立时会再次通知，教学实现即使一次没读空，也有机会在下一轮继续。

10. ET 版 epoll 服务器的修改点

从 LT 改到 ET，不需要推翻 epoll 接口，重点改三处：

1. 监听 socket 和普通连接都设置为非阻塞；
2. 新连接注册 `EPOLLIN | EPOLLET`；
3. 读写都循环到 `EAGAIN` 或输出缓冲区清空。

10.1 注册 ET

```
bool Add(int fd, uint32_t events, bool edge_triggered) {
    epoll_event ev = {0};
    ev.data.fd = fd;
    ev.events = events;

    if (edge_triggered) {
        ev.events |= EPOLLET;
    }

    return epoll_ctl(epoll_fd_, EPOLL_CTL_ADD, fd, &ev) == 0;
}
```

10.2 非阻塞读接口

```
bool RecvNoBlock(int fd, std::string* input) {
    char buffer[4096];
```

```
    for (;;) {
        ssize_t n = recv(fd, buffer, sizeof(buffer), 0);

        if (n > 0) {
            input->append(buffer, static_cast<size_t>(n));
            continue;
        }
        if (n == 0) {
            return false; // 对端关闭
        }
        if (errno == EINTR) {
            continue;
        }
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            return true; // 本轮读空
        }
        return false;
    }
}
```

注意，这个函数使用一个临时输入缓冲区累积本次读到的数据。真实协议层还需要把它追加到连接自己的输入缓冲区，因为一次 `recv` 可能只有半个请求，也可能包含多个请求。

10.3 非阻塞写接口

```
enum class SendResult {
    kDone,
    kWouldBlock,
    kError
};

SendResult SendNoBlock(int fd, const std::string& response) {
    size_t offset = 0;

    while (offset < response.size()) {
        ssize_t n = send(
            fd,
            response.data() + offset,
            response.size() - offset,
            MSG_NOSIGNAL
        );

        if (n > 0) {
```

```
            offset += static_cast<size_t>(n);
            continue;
        }
        if (n < 0 && errno == EINTR) continue;
        if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
            return SendResult::kWouldBlock;
        }
        return SendResult::kError;
    }

    return SendResult::kDone;
}
```

可以在这里返回“未发完”，更完整的 Reactor 会保留发送偏移和剩余缓冲区，并在 `EPOLLOUT` 到来后继续发送。

10.4 ET 事件分支

```
if (events[i].events & EPOLLIN) {
    std::string input;
    if (!RecvNoBlock(fd, &input)) {
        RemoveConnection(fd);
        continue;
    }

    HandleProtocolInput(fd, input);
}
```

这里的 `RecvNoBlock` 一定不能只调用一次 `recv`。在 ET 模式中，这次通知可能是本轮旧数据唯一一次被提醒的机会。

11. 为什么 ET 不一定“性能更高”

ET 减少重复通知，因此事件循环可以少做若干次“同一个 fd 仍然可读”的唤醒。但它的代价是：

- 必须循环读到 `EAGAIN`；
- 必须使用非阻塞 I/O；
- accept 必须循环；
- 发送需要完整状态机；

- 读写代码更复杂；
- 单个连接一次处理过久会影响其他连接的公平性；
- 协议解析仍需处理半包和粘包。

如果 LT 的实现每次都及时排空，两者在真实 I/O 成本上的差距可能没有想象中大。ET 更适合对大量连接和通知次数敏感、并且代码能够严格处理边沿的场景。

12. 连接关闭的推荐顺序

```
1. 从业务映射中找到连接
2. epoll_ctl DEL
3. 从 fd -> Connection 映射中删除
4. close(socket fd)
5. 延迟销毁 Connection 对象
```

先删除 epoll 关注关系，可以避免后续事件继续指向正在失效的 fd。先保留对象到当前事件处理结束，可以避免仍在分发同一轮事件时访问已释放对象。

fd 编号会被系统复用，因此不能把 fd 当作永久连接 ID。长期运行的服务通常需要额外的连接 ID 或引用管理。

13. 本篇检查清单

- 每个注册到 epoll 的 fd 都有明确的事件集合；
- `EPOLLERR` 和 `EPOLLHUP` 不被忽略；
- ET 连接使用非阻塞 I/O；
- ET 的读、accept、send 都有正确的循环和退出条件；
- `EAGAIN/EWOULDBLOCK` 不被当成连接错误；
- `EPOLLOUT` 按需开启；
- 输出数据可能只发送一部分；
- 关闭连接时先撤销事件，再释放资源；
- 事件数组只处理 `[0, n)`；
- 业务处理不会长时间阻塞事件循环。

14. 从服务器循环到 Reactor

一个可以工作的 epoll 服务器已经有了“事件循环”，但把所有分支写在 `main_loop` 中会越来越难维护：

- 每个连接有独立输入和输出缓冲区；
- 每个连接有自己的关闭状态；
- 协议层需要拆包、反序列化和序列化；
- 读、写、异常需要不同回调；
- 新增连接和普通连接的管理逐渐分离；
- 多线程后还要明确连接归属。

Reactor 的任务就是把这些分散在循环中的状态和逻辑重新组织成对象，同时保留已经建立的 epoll 事件模型。